



Figure 1: Directory structure

objdump and *readelf*, amongst others.

- **gas** contains the GNU assembler. Target-specific assembler code is held in the *config* sub-directory.
- **ld** contains the GNU linker. Target-specific linker files are held in sub-directories.
- **gprof** contains the GNU profiler. This program does not have any target-specific code.
- **gold** contains the new GNU linker. This is a new linker being created to replace LD. At the moment it is still in development.
- **elfcpp** contains *elfcpp*, a C++ library for reading and writing ELF information. It is currently only used by the GOLD linker.

In addition, there are a couple of other directories that you can find at the top level of a binutils source release. They are used in the binutils build process, but are not part of the binutils project:

- **intl** contains the GNU *gettext* library from *gettext*.
- **libiberty**: Before POSIX and glibc, this was a GNU project to provide a set of standard functions. It lives on in binutils. Most valuable are its free store management and argument parsing functions.

Main functional areas and data structures

Binary file description

Binary file description (BFD) is a package that allows applications to use the same routines to operate on object files, whatever the object file format. A new object file format can be supported simply by creating a new BFD backend and adding it to the library.

The BFD library backend creates a number of data structures describing the data held in a particular type of object file. Ultimately, a unique enumerated constant (of type *enum bfd_architecture*) is defined for each individual architecture. This constant is then used to access the various data structures associated with that particular architecture.

In the case of the Compact-RISC, 16-bit implementation (which may be a COFF or ELF binary), the enumerated constant is *bfd_cr16_arch*. This can be used to access various structures, for example:

```
const bfd_arch_info_type bfd_cr16_arch =
{
    16, /* 16 bits in a word. */
    32, /* 32 bits in an address. */
    8, /* 8 bits in a byte. */
    bfd_arch_cr16, /* enum bfd_architecture arch. */
    bfd_mach_cr16, /* Machine value, used to distinguish between cr16 variants. */

```

Further sources of information

Written documentation

The main user guides for binutils tools provide a great deal of context about how they are intended to work. The document on the binutils internals is essential reading, before and during any porting exercise. It is not complete, nor is it always up to date, but it is the first place to look for an explanation on what a particular function does.

The binutils tools rely upon a separate specification of the Binary File Descriptor for each architecture. This has its own comprehensive user guide.

The main binutils tools code base is generally well commented, particularly in the headers for the major interfaces. Inevitably, this emerges as the definitive place to find out exactly how a particular function is expected to behave.

Other information channels

The main website for binutils tools is at sourceware.org/binutils. The binutils developer community communicates through the binutils mailing lists. These are always good places to find solutions to a problem.

The main mailing list for discussions is binutils@sourceware.org, although for a detailed understanding, reading the bug reporting mailing list bug-binutils@sourceware.org is also recommended. See the main binutils website for details about subscribing to these mailing lists.

```

"cr16", /* Architecture name (short version). */
"cr16", /* Architecture name (long version). */
1, /* Section alignment power. */
TRUE, /* True if this is the default machine for the
architecture. */
bfd_default_compatible, /* Function to call to determine if two different
architectures are compatible. */
bfd_default_scan, /* Function to call to determine if a given string
matches this architecture. */
NULL, /* Pointer to the next CR16 machine architecture. */
}

```

This particular structure is defined in a file called *cpu-target.c* in the *bfd* directory.

The file *file_format-<target>.c* (for example, *elf32-cr16.c*) is used to provide target-specific support for the given file format and architecture. At the very least, it provides the following information:

1. A *reloc_map* array that maps BFD relocation enumerations into a target-specific relocation type.
2. A *reloc_howto* array with target-specific relocation details. Here is an example array entry from the cr16 port:

```

(R_CR16_NONE, /* Type. */
0, /* Rights shift. */
2, /* Size. */
32, /* Bitsize. */
FALSE, /* PC relative. */

```